

1. Infrastructure Foundation

- **SSH & Connectivity:** The Secure Shell server is enabled on pfSense. The Wazuh server generates a new RSA key pair, and ssh-copy-id is used to copy the public key to the firewall (admin@192.168.1.1), establishing secure, passwordless authentication for automated responses.
- **Firewall Enforcement:** An alias named blocked_ips is created in pfSense to act as a container for IPs that need to be blocked. A firewall rule is then established on the LAN interface to block all traffic coming from the hosts defined in this alias. Another rule is configured on the OPT1 interface to manage traffic flow based on specified conditions.

2. ML Correlation

The engine acts as a Hybrid Detection and Correlation System, solving the limitations of traditional SIEMs by combining rule-based detection with behavioral analysis. It follows this workflow:

- **Step 1:** Data Acquisition: The system monitors /var/ossec/logs/alerts/alerts.json for all incoming Wazuh alerts.
- **Step 2:** Field Extraction: Essential metadata (Source IP, Target Host, Logon Type, Command Line) is parsed for field-aware analysis.
- **Step 3:** Signal & Stage Detection: Events are converted into behavioral signals and classified into Attack Stages (e.g., Recon, Initial Access, Lateral Movement) to track the progression of an attack chain.
- **Step 4:** Session Correlation: Related events are grouped via a chain_key (linking Attacker IP, Target Host, and Session ID), allowing the system to analyze the *entire attack session* rather than independent events.
- **Step 5:** Feature Engineering: The system extracts eight core behavioral features for the ML model:
 - Direct: "Velocity" (speed of attack), "Unique_rules" (diversity of rules triggered), "Time_delta" (time gap between events), "Severity" (Wazuh rule level), and "Is_sequence" (whether events occur in sequence).
 - Derived: "Vel_sev" (velocity × severity), "Vel_time" (velocity × time_delta), and "rule_time" (unique_rules × time_delta).
- **Step 6:** ML Base Probability: These features are fed into a RandomForestClassifier (100 trees, max depth 14, class-balanced), which processes them to output a predict_proba() base attack probability.

3. Contextual Scoring & Risk Calculation

After the ML model generates the base probability, the system performs Contextual Scoring to calculate the final risk score:

- Contextual Correlation Logic: The risk score is dynamically adjusted by analyzing overall attack behavior, such as repeated failed logins, NTLM admin access, privilege escalation, and suspicious command execution.
- Risk Smoothing: The system uses SMOOTHING_NEW and SMOOTHING_OLD parameters to prevent volatile score fluctuations, maintaining continuity in attack tracking.
- Final Risk Decision: The combination of the ML base probability and the contextual/historical behavior creates the Final Dynamic Risk Score.

Dataset	1 Million samples (400k Attack, 600k Normal) with "hard negatives" added.
Model	RandomForestClassifier (n_estimators=100, max_depth=14).
Evaluation	Threshold 0.63 used for classification.
Performance	Accuracy (0.94), Recall (0.9129).

Secure Shell

Secure Shell Server

☒ Enable Secure Shell

SShd Key Only

Password or Public Key

When set to *Public Key Only*, SSH access requires authorized keys and these keys must be configured for each [user](#) that has been granted secure shell access. If set to *Require Both Password and Public Key*, the SSH daemon requires both authorized keys **and** valid passwords to gain access. The default *Password or Public Key* setting allows either a valid password or a valid authorized key to login.

Allow Agent Forwarding

☒ Enables ssh-agent forwarding support.

SSH port

22

Note: Leave this blank for the default of 22.

Figure 1 Configuring SSH on pfSense to allow secure remote connections for automated response functionality.

```
wazuh-user@wazuh-server:~ X + v - □ X
[wazuh-user@wazuh-server ~]$ sudo ssh-keygen -t rsa -f /root/.ssh/id_rsa -N ""
Generating public/private rsa key pair.
/root/.ssh/id_rsa already exists.
Overwrite (y/n)? y
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:g9TuZPaBm0MS7Sey0PS8I/c4WfZ44lUeJEYrRTcemYA root@wazuh-server
The key's randomart image is:
+----[RSA 3072]-----+
|
|  o=.+o
|  o  Eo +oo
| + o . + o
| + B . o o
| . = S o o
| . X Oo. o .
| o O+.o. .
| oo=o.o.o
| .ooo
+-----[SHA256]-----+
[wazuh-user@wazuh-server ~]$ sudo ssh-copy-id -i /root/.ssh/id_rsa.pub admin@192.168.1.1
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/root/.ssh/id_rsa.pub"
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that
are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is
to install the new keys
(admin@192.168.1.1) Password for admin@pfSense.home.arpa:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh 'admin@192.168.1.1'"
and check to make sure that only the key(s) you wanted were added.

[wazuh-user@wazuh-server ~]$ sudo ssh admin@192.168.1.1
VirtualBox Virtual Machine - Netgate Device ID: 62a23a718a4f12d08b42

*** Welcome to pfSense 2.6.0-RELEASE (amd64) on pfSense ***

WAN (wan)      -> em0      -> v4/DHCP4: 10.0.2.15/24
LAN (lan)      -> em1      -> v4: 192.168.1.1/24
OPT1 (opt1)    -> em2      -> v4: 192.168.56.100/24
OPT2 (opt2)    -> em3      -> v4: 192.168.57.1/24

0) Logout (SSH only)          9) pfTop
1) Assign Interfaces          10) Filter Logs
2) Set interface(s) IP address 11) Restart webConfigurator
3) Reset webConfigurator password 12) PHP shell + pfSense tools
4) Reset to factory defaults    13) Update from console
5) Reboot system              14) Disable Secure Shell (sshd)
6) Halt system                15) Restore recent configuration
7) Ping host                  16) Restart PHP-FPM
8) Shell
```

Figure 2 SSH key generation and configuration for establishing secure communication between Wazuh server and pfSense firewall

Firewall / Aliases / Edit ?

Properties

Name

blocked_ips

The name of the alias may only consist of the characters "a-z, A-Z, 0-9 and _".

Description

A description may be entered here for administrative reference (not parsed).

Type

Host(s)

Host(s)

Hint

Enter as many hosts as desired. Hosts must be specified by their IP address or fully qualified domain name (FQDN). FQDN hostnames are periodically re-resolved and updated. If multiple IPs are returned by a DNS query, all are used. An IP range such as 192.168.1.1-192.168.1.10 or a small subnet such as 192.168.1.16/28 may also be entered and a list of individual IP addresses will be generated.

IP or FQDN

192.168.1.200

Description

Figure 3 Configuration of firewall aliases in pfSense, specifying blocked IPs for automated security responses.

Firewall / Rules / Edit ≡ 📊 📄 ?

Edit Firewall Rule

Action

Block

Choose what to do with packets that match the criteria specified below.
Hint: the difference between block and reject is that with reject, a packet (TCP RST or ICMP port unreachable for UDP) is returned to the sender, whereas with block the packet is dropped silently. In either case, the original packet is discarded.

Disabled

☐ Disable this rule

Set this option to disable this rule without removing it from the list.

Interface

LAN

Choose the interface from which packets must come to match this rule.

Address Family

IPv4

Select the Internet Protocol version this rule applies to.

Protocol

Any

Choose which IP protocol this rule should match.

Source

Source

☐ Invert match

Single host or alias

blocked_ips

/

Destination

Destination

☐ Invert match

any

Destination Address

/

Figure 4 Firewall rule configuration in pfSense to block traffic from specified IPs in the LAN network.


```
wazuh-user@wazuh-server:~ X + v

[wazuh-user@wazuh-server ~]$ tail -f build_log.txt
nohup: ignoring input
--- STAGE 1: PROCESSING AIT-ADS JSON ---
Reading fox_wazuh.json...
Processed 5000 lines...
Processed 10000 lines...
Processed 15000 lines...
Processed 20000 lines...
Processed 25000 lines...
Processed 30000 lines...
Processed 35000 lines...
Processed 40000 lines...
Processed 45000 lines...
Processed 50000 lines...
Processed 55000 lines...
Processed 60000 lines...
Processed 65000 lines...
Processed 70000 lines...
Processed 75000 lines...
Processed 80000 lines...
Processed 85000 lines...
Processed 90000 lines...
Processed 95000 lines...
Processed 100000 lines...
Processed 105000 lines...
Processed 110000 lines...
Processed 115000 lines...
Processed 120000 lines...
Processed 125000 lines...
Processed 130000 lines...
Processed 135000 lines...
Processed 140000 lines...
Processed 145000 lines...
Processed 150000 lines...
Processed 155000 lines...
Processed 160000 lines...
Processed 165000 lines...
Processed 170000 lines...
Processed 175000 lines...
Processed 180000 lines...
Processed 185000 lines...
Processed 190000 lines...
Processed 195000 lines...
Processed 200000 lines...
Processed 205000 lines...
Processed 210000 lines...
Processed 215000 lines...
Processed 220000 lines...
Processed 225000 lines...
Processed 230000 lines...
Processed 235000 lines...
Processed 240000 lines...
Processed 245000 lines...
Processed 250000 lines...
Processed 255000 lines...
```

Figure 7 AIT-ADS Feature Extraction process.

```

import pandas as pd
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
import pickle

print("---- [1/4] LOADING DATASET ----")
df = pd.read_csv('ait_dataset.csv')
df.dropna(inplace=True)

X = df[['velocity', 'unique_rules', 'time_delta', 'severity', 'is_sequence']]
y = df['label']

print("---- [2/4] SPLITTING DATA ----")
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

print("---- [3/4] TRAINING RANDOM FOREST ----")
model = RandomForestClassifier(
    n_estimators=100,
    max_depth=12,
    min_samples_split=5,
    n_jobs=-1,
    verbose=1,
    class_weight='balanced'
)

model.fit(X_train, y_train)

print("---- [4/4] EVALUATING MODEL ----")
accuracy = model.score(X_test, y_test)
print(f'\n[RESULT] Final Model Accuracy: {accuracy*100:.2f}%')

print('\n[RESULT] Confusion Matrix:')
y_pred = model.predict(X_test)
print(confusion_matrix(y_test, y_pred))

print('\n[RESULT] Classification Report:')
print(classification_report(y_test, y_pred))

print('\n[ANALYSIS] Feature Importance:')
importances = model.feature_importances_
for name, importance in zip(X.columns, importances):
    print(f'{name}: {importance:.4f}')

with open('model.pkl', 'wb') as f:
    pickle.dump(model, f)

print("\nModel saved as model.pkl")

```

Figure 8 Training the Random Forest model using system event data


```
wazuh-user@wazuh-server:~ X + v
EOF
[wazuh-user@wazuh-server ~]$ cat << 'EOF' > evaluate_model.py
import pandas as pd
import joblib
from sklearn.metrics import classification_report, confusion_matrix

model = joblib.load('threat_model.pkl')
X_test = pd.read_csv('X_test.csv')
y_test = pd.read_csv('y_test.csv')

probs = model.predict_proba(X_test)[:,:1]
y_pred = (probs > 0.63).astype(int)

print(classification_report(y_test, y_pred, target_names=['Normal', 'Attack']))

cm = confusion_matrix(y_test, y_pred)
tn, fp, fn, tp = cm.ravel()

fpr = fp / (fp + tn)
recall = tp / (tp + fn)

print(cm)
print(f"FPR: {fpr:.4f}")
print(f"Recall: {recall:.4f}")
EOF
[wazuh-user@wazuh-server ~]$ python3 generate_dataset.py
python3 train_model.py
python3 evaluate_model.py
```

	precision	recall	f1-score	support
Normal	0.94	0.97	0.95	120000
Attack	0.95	0.91	0.93	80000
accuracy			0.94	200000
macro avg	0.94	0.94	0.94	200000
weighted avg	0.94	0.94	0.94	200000

```
[[115839  4161]
 [ 6969 73031]]
FPR: 0.0347
Recall: 0.9129
```

Figure 9 Evaluation of the machine learning model's performance with precision, recall, and F1-score metrics

